

Agent 的概念、原理与构建模式

—— 从零打造一个简化版的 Claude Code

整理自 B 站「马克的技术工作坊」视频教程

一、引言：大模型的「能力边界」

在正式理解 Agent 之前，需要先想清楚一个问题：今天的大模型（如 GPT-4o、DeepSeek 等）虽然回答问题很厉害、逻辑也很强，但它们有一个绕不开的限制——

大模型无法感知，也无法改变外部世界。

视频里举了一个很直观的例子：

- 你让 GPT-4o 帮你写一个贪吃蛇游戏，它可以给出完整的代码；
- 但把代码「写入到文件」这件事，仍然得你自己动手；
- 更常见的场景是：你本地其实已经有一份贪吃蛇代码，只是想让模型基于它改写、加一点功能——如果不主动把代码复制给模型，它是查不到这些文件的。

「查不到已有文件」「没法自己写文件」「改不了外部环境」，这就是大模型无法感知/改变外界环境的典型体现。那么问题来了——怎么破？

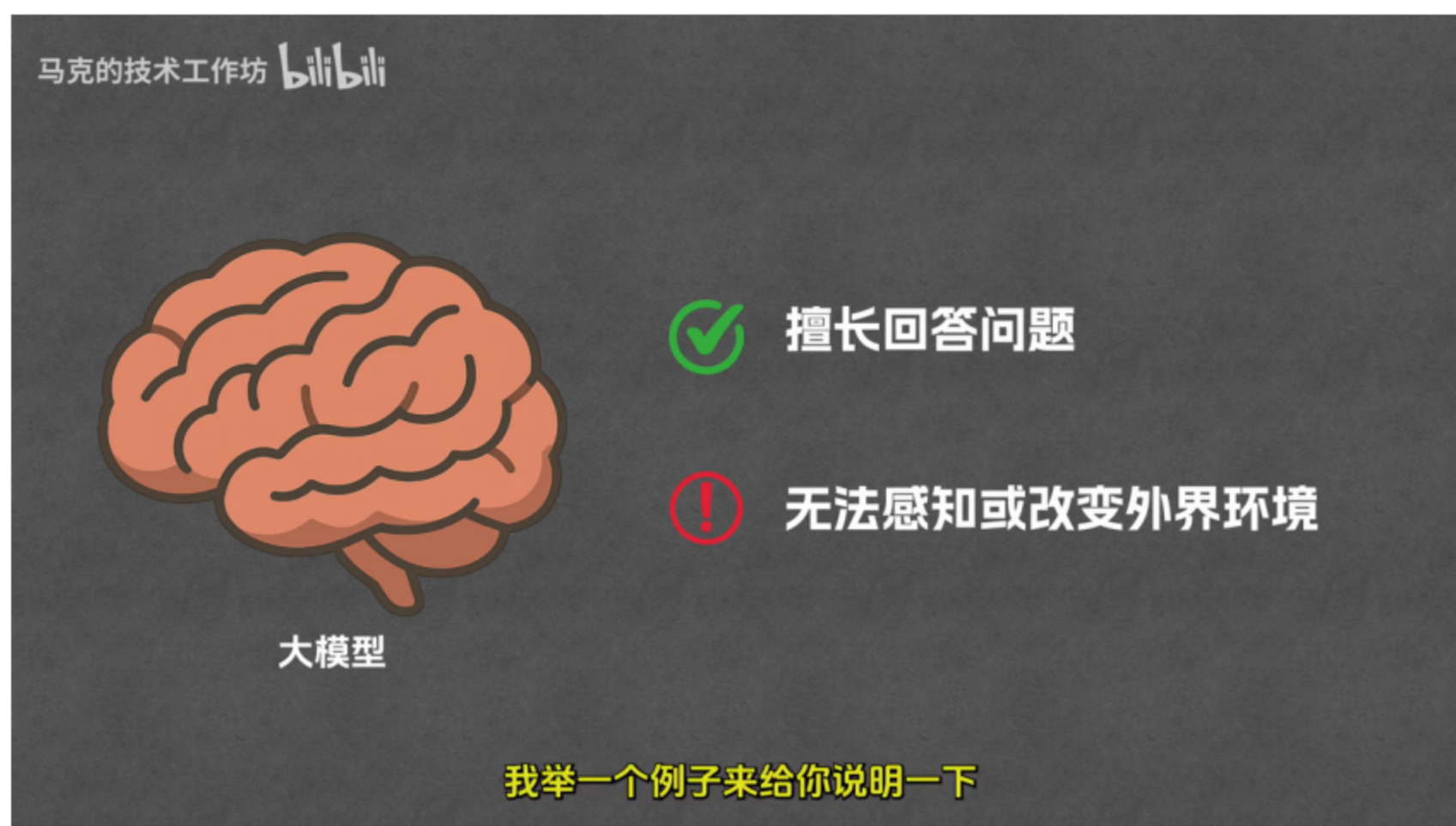


图 1-1 大模型能生成代码，却无法自己读写文件、感知环境

二、什么是 Agent (AI 智能体)

解决思路其实很简单：给大模型接上「工具 (Tool)」。

- 读写文件内容的工具；
- 查看文件列表的工具；

- 运行终端命令的工具。

工具就像是大模型的「感官和四肢」。有了工具，大模型就能自己查询已有文件、自己写入代码、自己运行程序，整个过程不需要人插手，完全自动化。

把「一个大模型」和「一堆工具」组装起来，变成一个能感知和改变外界环境的智能程序——这就是 Agent（智能体）。

视频里有一个很形象的对比：大模型常被画成一个「大脑」图标，而 Agent 则被画成一个「机器人」图标。因为 Agent 有了感官和四肢，能自己独立做事，就像一个机器人。

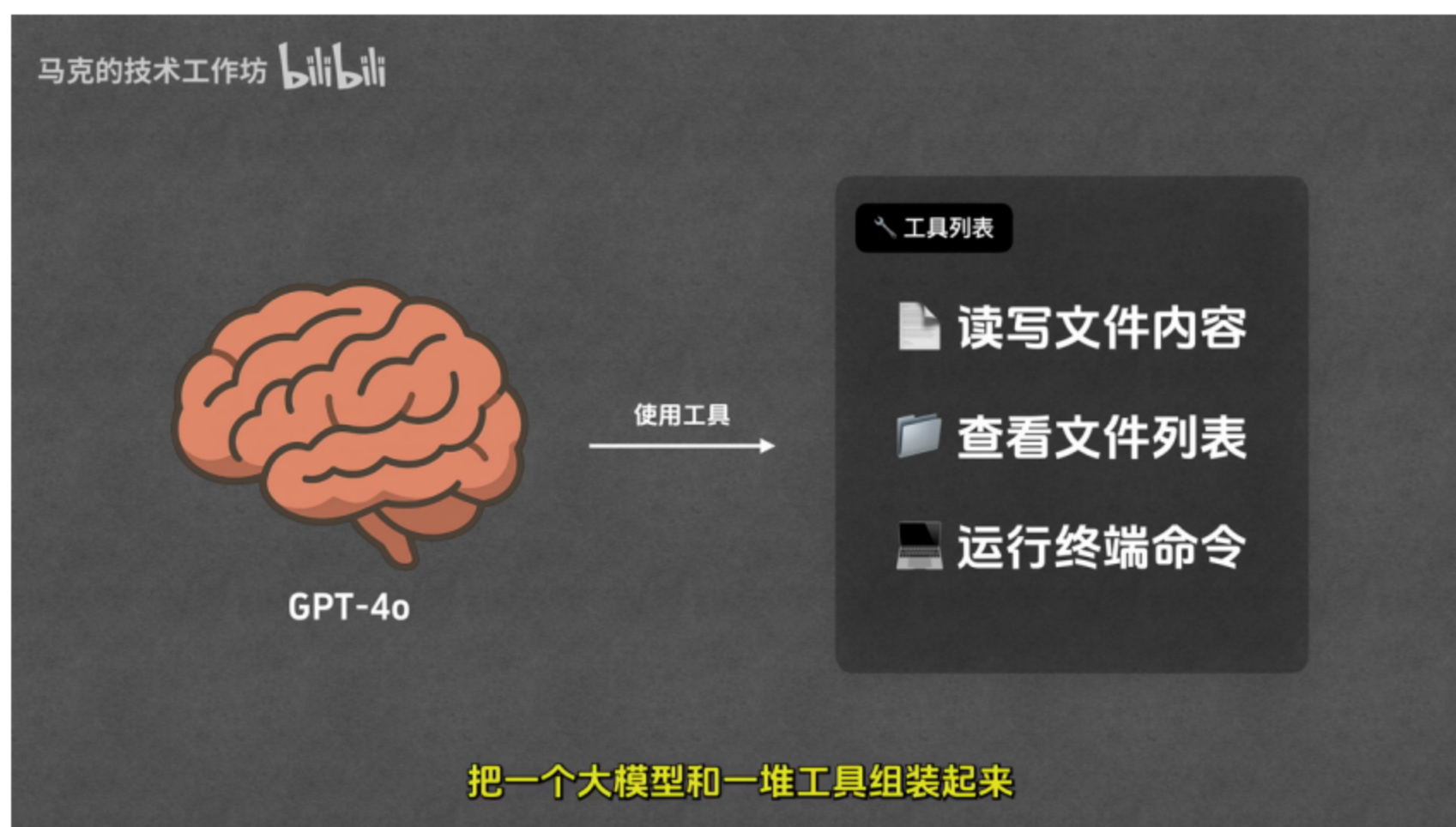


图 2-1 大脑（大模型）vs 机器人（Agent）：工具是 Agent 的感官与四肢

三、Agent 的常见类型

Agent 不止一种，擅长的领域各不相同。视频里重点提了三类：

- 编程类 Agent：比如 Cursor、以及接下来要动手实现的简化版 Claude Code，提交任务后它会调用大模型和工具自动写代码；
- 做 PPT 的 Agent：自动生成演示文稿；
- 深度搜索 Agent：比如曾经很火的 Manus，用户让它「比较几款手机的拍照/性能」，它会生成执行计划、搜索浏览网页、最后把报告整理成页面。

也就是说，Agent 的「类型有很多，擅长的领域也各不相同」，但其底层思想是一致的：大模型 + 工具 + 自动运行。

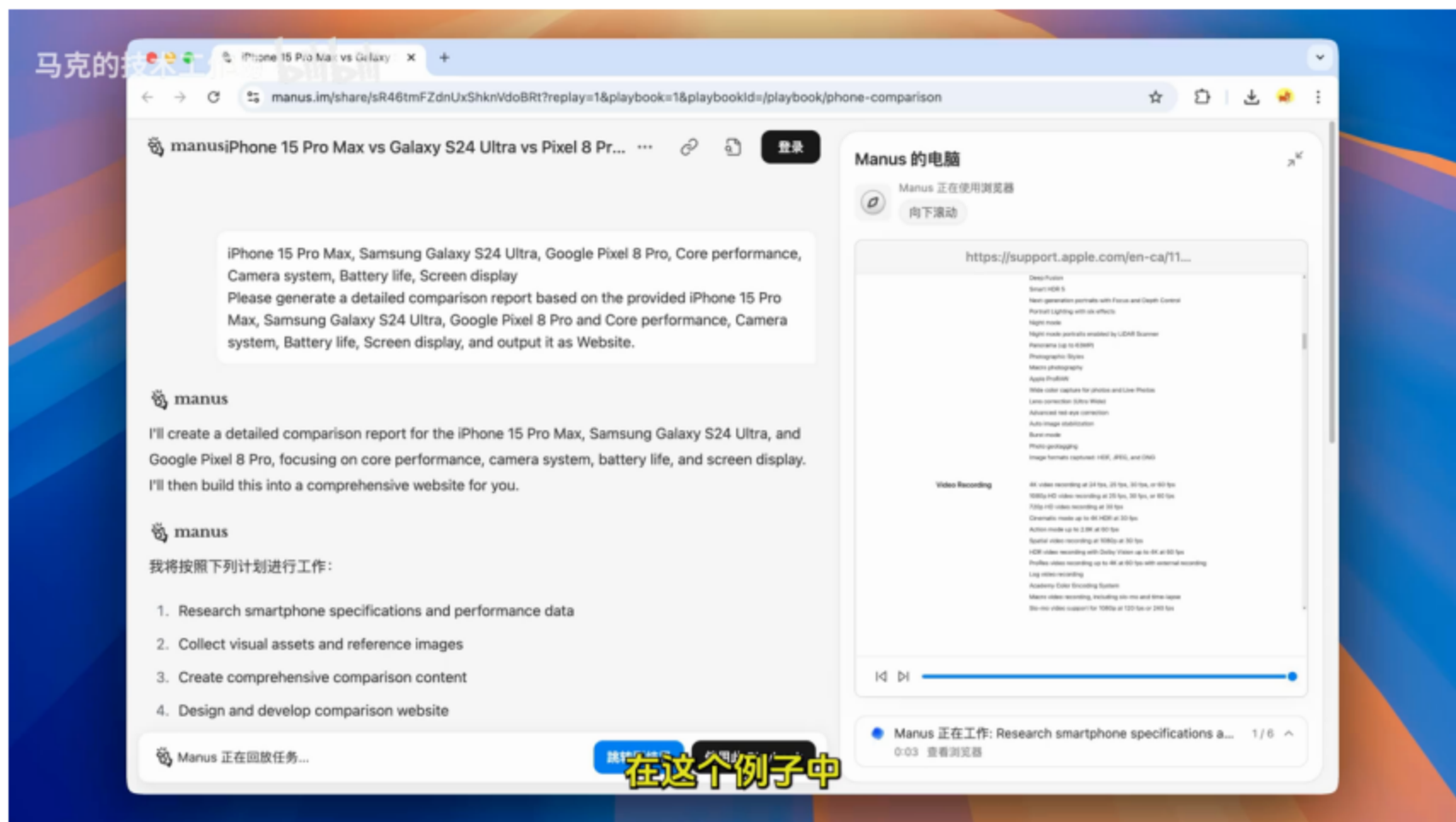


图 3-1 Agent 类型举例：从 Cursor 到 Manus 等

四、Agent 的核心运行模式：ReAct

Agent 的运行模式有很多种，目前最著名、使用最广泛的是 ReAct（读作「瑞·阿克特」）。

ReAct 是 Reasoning + Acting 的缩写，即「思考与行动」。

它最早由 2022 年 10 月的一篇论文提出，虽然已过去近 3 年，但至今仍是使用最广的 Agent 运行模式——学习 Agent 实现原理，绝对绕不开它。

ReAct 的核心流程是一个循环：

- Thought（思考）：Agent 先思考，判断是否要调用工具；
- Action（行动）：如果需要，就调用合适的工具，如读取/写入文件；
- Observation（观察）：查看工具的执行结果；
- 循环：观察之后继续思考，再判断是否需要调用工具，重复「思考→行动→观察」；
- Final Answer（最终答案）：当 Agent 认为信息足够、无需再调用工具时，直接给出最终结论。

记住 Thought / Action / Observation / Final Answer 这四个词，后面代码实现里会反复用到。

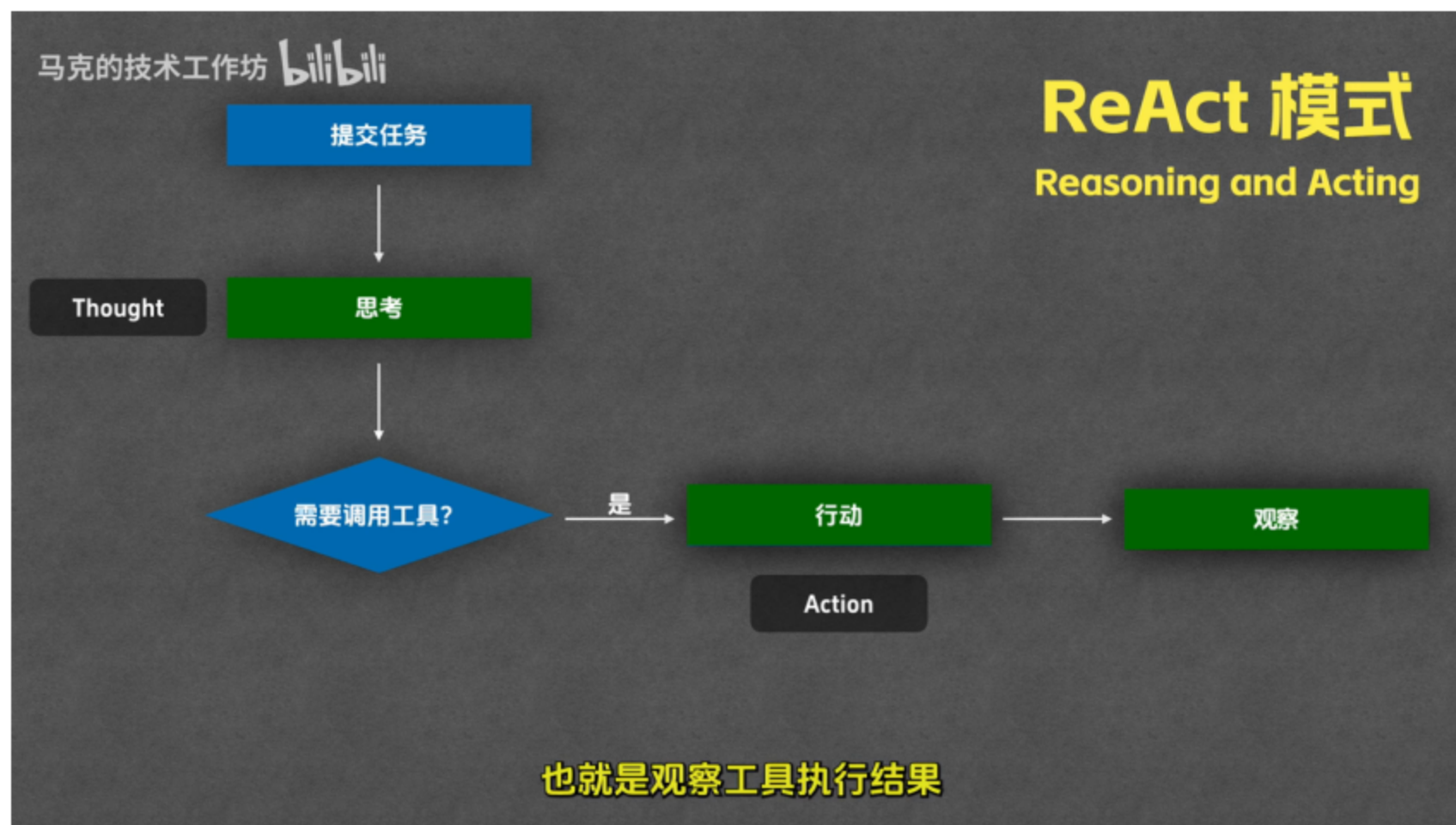


图 4-1 ReAct 运行模式流程图：思考 → 行动 → 观察 →（循环）→ 最终答案

五、ReAct 的奥秘：系统提示词（System Prompt）

一个关键问题：为什么模型拿到用户问题后，会「先思考再行动」，而不是直接行动？是因为模型被专门训练成这样吗？

并不是。奥秘几乎全部集中在「系统提示词」上。

系统提示词是和用户问题一起送给模型的提示词，它规定了模型的角色、运行时要遵守的规则、以及各种环境信息。只要你在系统提示词里写「你的回答必须包含两个 XML 标签：question 存放用户问题、answer 存放你的回答」，模型就会照此规范输出。

要让模型按 ReAct 模式运行，系统提示词会更复杂。视频里给出了一份典型的系统提示词，大致包含五个部分：

- ① 职责描述：把任务分解为多步，每步先用 Thought 思考，再用 Action 调用工具，结果通过 Observation 返回，直到能给出 Final Answer；
- ② 示例：给出几个完整样例（如「埃菲尔铁塔多高」→ Thought → Action 调用 get_height → Observation → Final Answer）；
- ③ 可用工具：列出读取文件、写入文件、运行终端命令等工具；

- ④ 注意事项：对危险操作（如运行终端命令）给出约束；
- ⑤ 环境信息：当前操作系统、工作目录、目录下的文件列表等。

这段系统提示词，相当于给模型安排了一个「迷你剧本」，它会严格按剧本一步步走完。

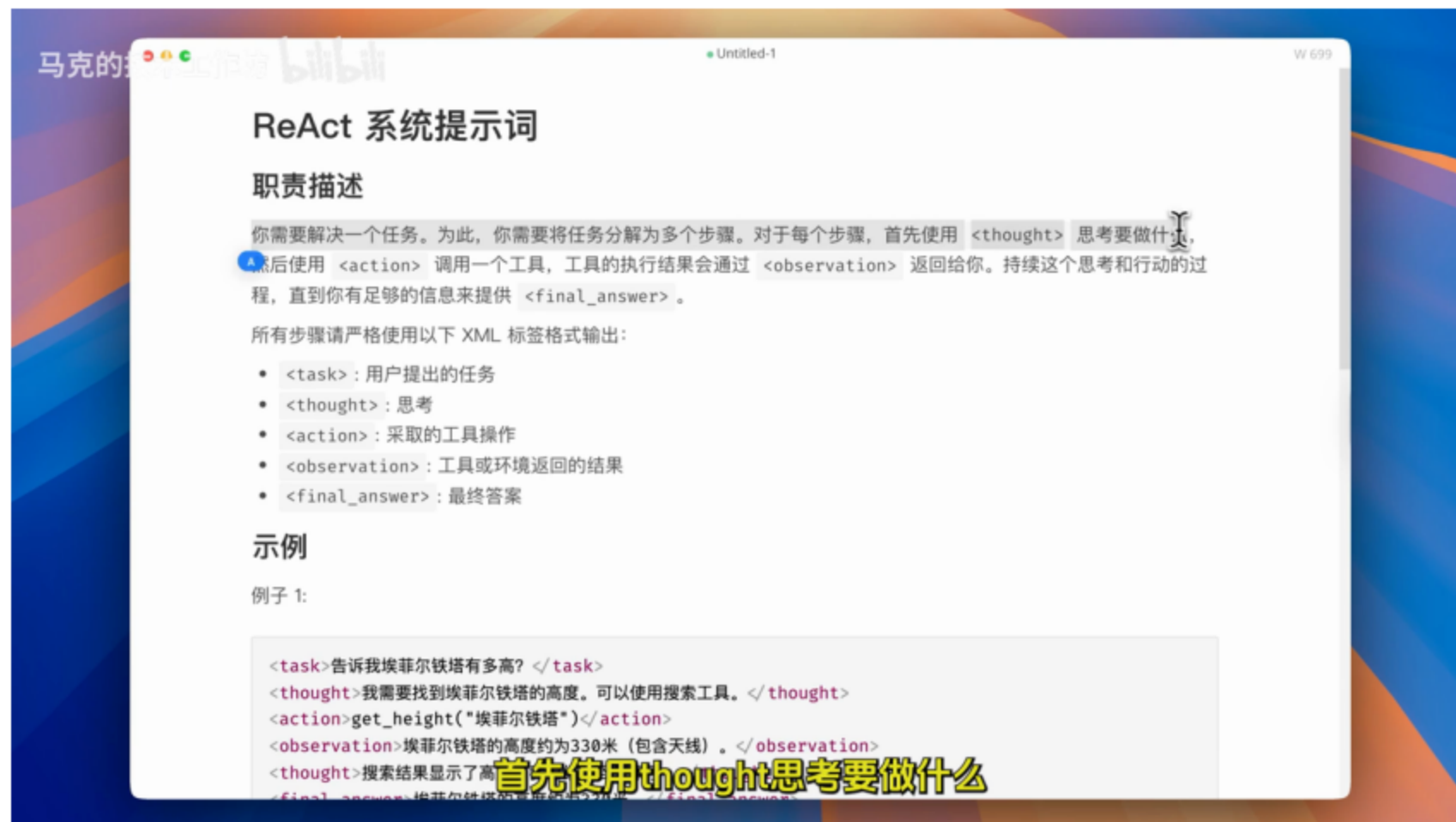


图 5-1 系统提示词五个组成部分：职责描述 / 示例 / 可用工具 / 注意事项 / 环境信息

六、实战：用 DeepSeek 模拟一次 ReAct

视频里用 DeepSeek 现场演示了一遍 ReAct 流程（注：DeepSeek 没有提供单独提交系统提示词的入口，所以把系统提示词和用户任务合在一起作为一条消息提交，多数情况下模型依然能按预期运行）。

任务：写一个贪吃蛇游戏，用 HTML/CSS/JS 实现，分别放在不同文件中。提交后可以看到：

- 第一轮：模型先在 <thought> 中思考，再用 <action> 请求调用 write_to_file 写入 index.html，我们回复 <observation> 「写入成功」；
- 第二轮：模型再次 Thought → Action 请求写入 CSS 文件，仍回复「写入成功」；
- 第三轮：模型 Thought → Action 写入 JS 文件；
- 收尾：三个文件都写完后，模型返回 Thought + Final Answer，流程结束。

这里有个非常关键的区别要记住：

大模型本身「不能」调用工具，它只能「请求」调用工具（注意是请求二字）。真正去执行 `write_to_file` 函数的是 Agent 的「工具调用组件」。刚才我们用人工回复 `observation` 来模拟，而真正运行 Agent 时，工具调用组件会自动执行并把结果塞回给模型。

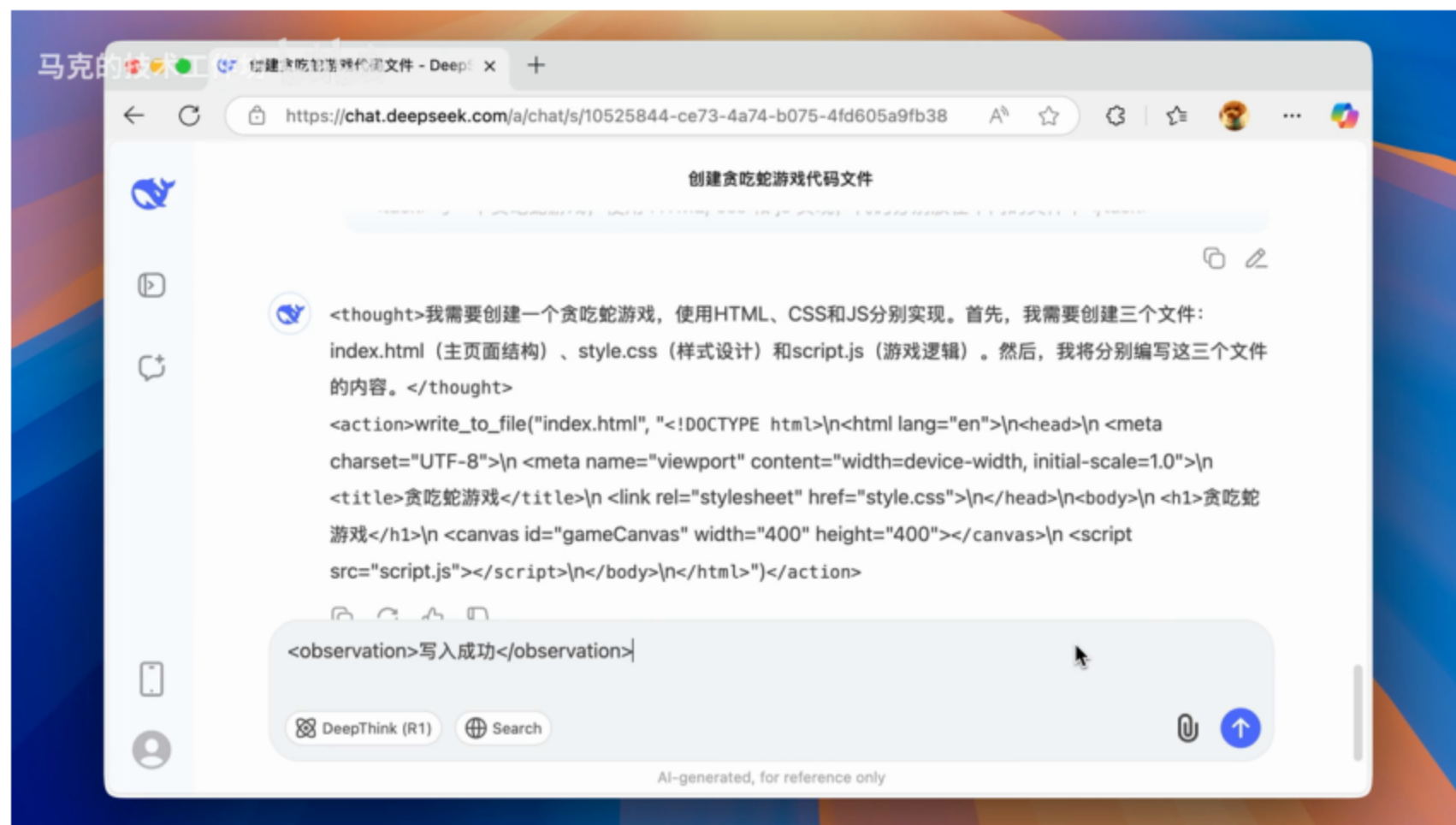


图 6-1 用 DeepSeek 模拟 ReAct: 模型按 Thought → Action → Observation 节奏推进

七、动手实现：一个简化版 Claude Code（ReAct Agent）

在「系统提示词 + 配套代码」的基础上，就能搭出一个真正可用的 ReAct Agent。视频作者已经把它开源在 GitHub 上，并现场演示+逐行讲代码。

7.1 运行效果演示

进入项目目录后，用命令启动 Agent：

```
uv run agent.py snake
```

其中 `snake` 是脚本的第一个参数，告诉 `agent.py` 要操作的项目目录就是 `snake` 文件夹（里面原本是空的）。启动后输入任务「写一个贪吃蛇游戏，用 HTML/CSS/JS 实现，代码分别放在不同文件中」，Agent 便开始运行。

运行结果同样是 Thought / Action / Observation 三段式，写入 `index.html` → `css` → `js`，最后给出 Final Answer。注意这次的「写入成功」是真实执行了 `write_to_file` 工

具，不是模拟。打开 index.html，贪吃蛇能跑、能吃到方块、左上角有分数——完全可以作为简化版 Claude Code 使用。



图 7-1 运行 uv run agent.py snake 的真实输出：Thought/Action/Observation 循环

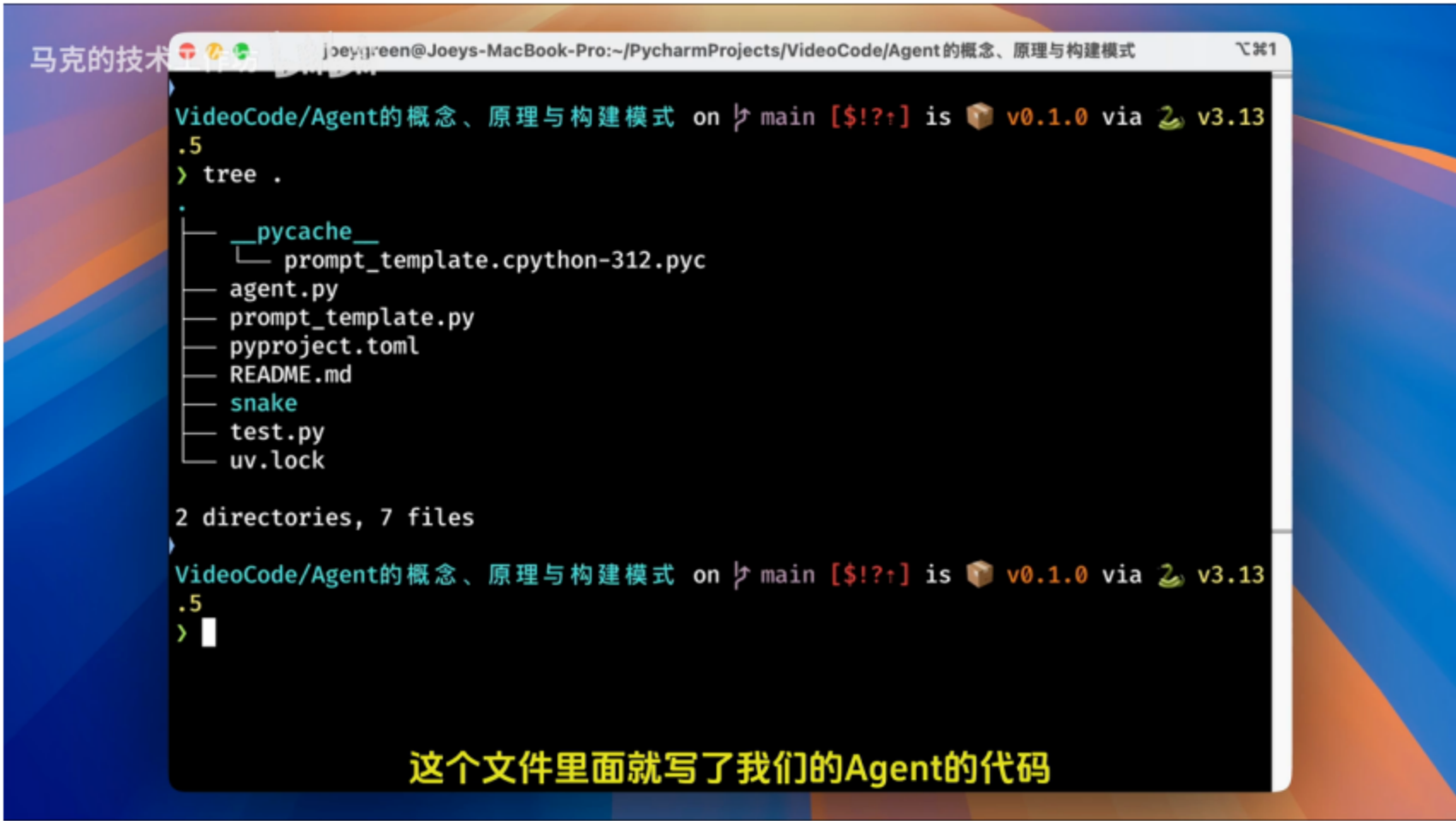


图 7-2 Agent 自动生成的贪吃蛇游戏，可直接在浏览器运行

7.2 核心代码：入口 main()

项目里真正需要关注两个东西：agent.py（Agent 逻辑）和 snake 文件夹（产物）。入口代码如下：

- project_directory: 传给脚本的第一个参数（snake 文件夹）；
- tools: 可用工具列表，给出三个——读取文件、写入文件、运行终端命令；
- 实例化 ReactAgent，传入 tools、模型（GPT-4o）、项目目录；
- 提示用户输入任务，调用 agent.run(任务) 启动 Agent。

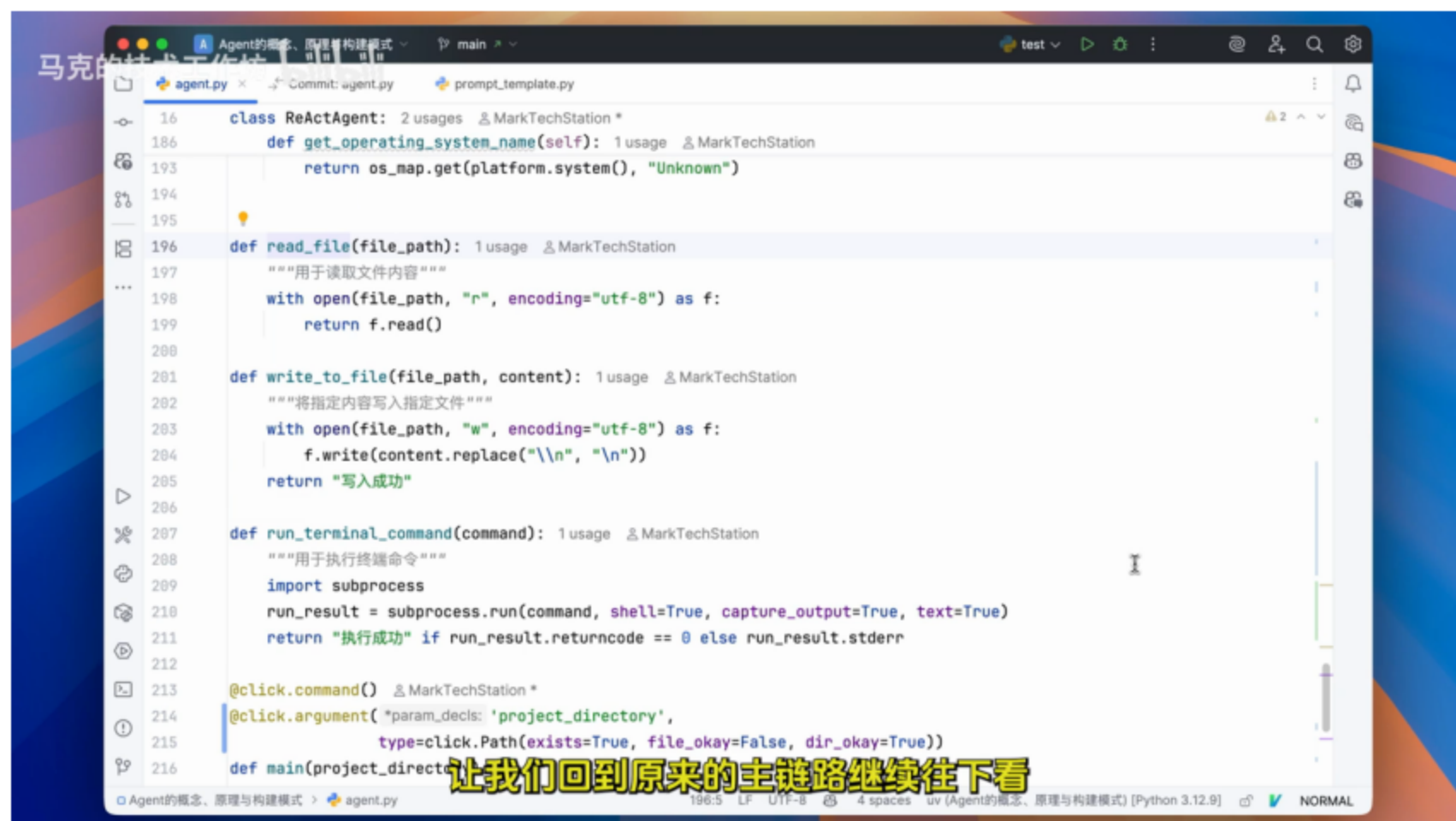


图 7-3 入口函数：构造工具列表 → 实例化 ReactAgent → run(任务)

7.3 核心类：ReactAgent

ReactAgent 类构造时接收三个参数：工具列表、要用的模型、项目目录。它还有一个 agent 主程序里负责串联流程的逻辑——也就是下面要讲的 run 函数。

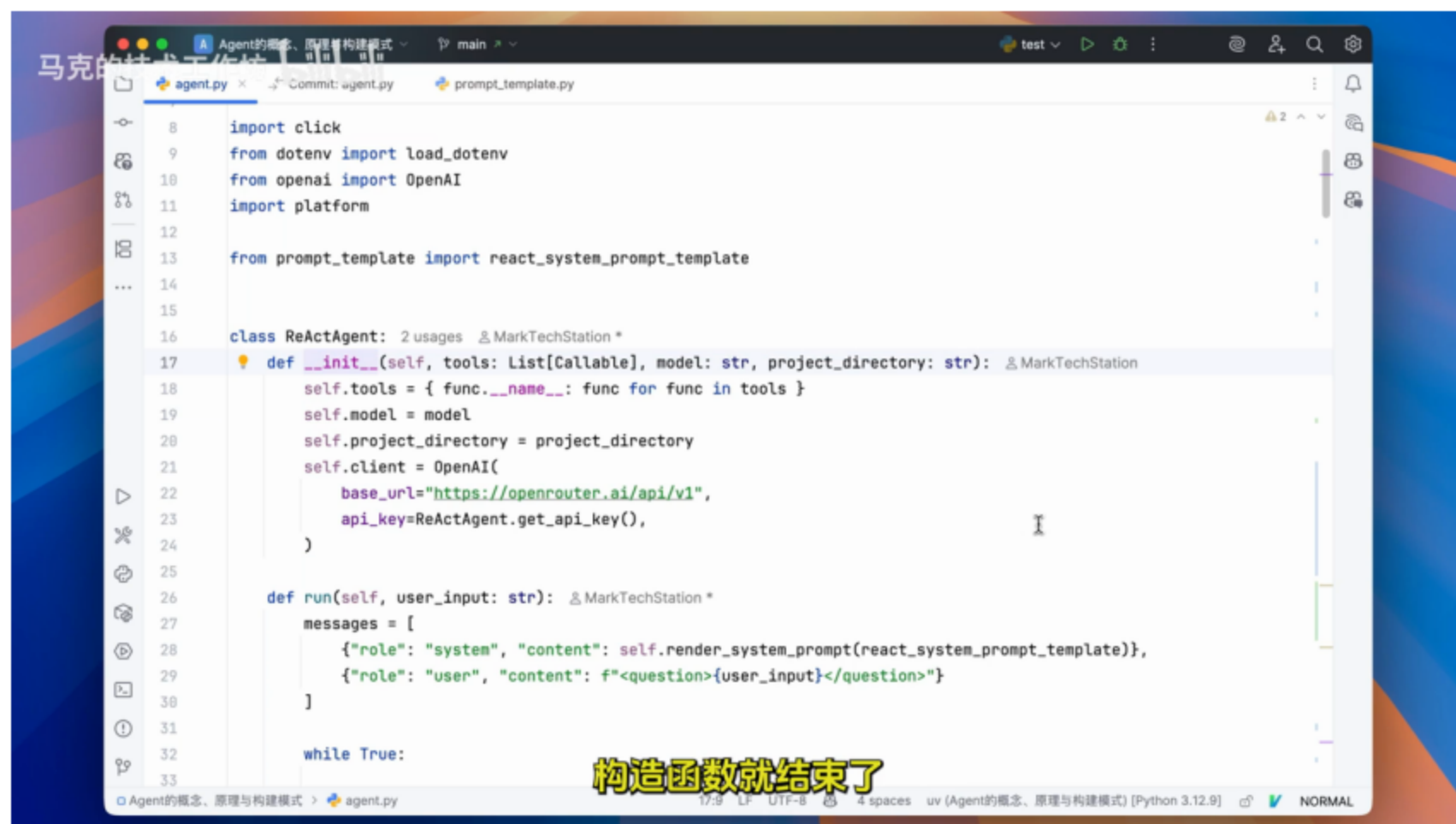


图 7-4 ReactAgent 类：构造工具/模型/目录/客户端

7.4 核心逻辑：run() 的 while 循环

run(任务) 是 ReactAgent 的核心调用，大致做了这些事：

- 构建 message 列表：含系统提示词（由 render_system_prompt 渲染，填入工具列表、操作系统、文件列表等占位符）和用户问题；
- 调用 call_model 拿到模型结果，提取 <thought> 打印；
- 检测 <thought> 之后是否是 Final Answer：是则结束；否则一定包含 <action>，解析出函数名与参数；
- 若是运行终端命令的工具，会先询问用户是否继续（因为危险）；
- 执行工具背后的函数，把结果放进 <observation>，再加入 message 列表；
- 回到循环开头，带着新结果再次请求模型。

一句话总结这个 while 循环：请求模型 → 提取 thought → 检测 final answer → 提取并执行 action → 循环，直到模型返回 Final Answer。回想一下，这正是前面讲的 ReAct 流程。

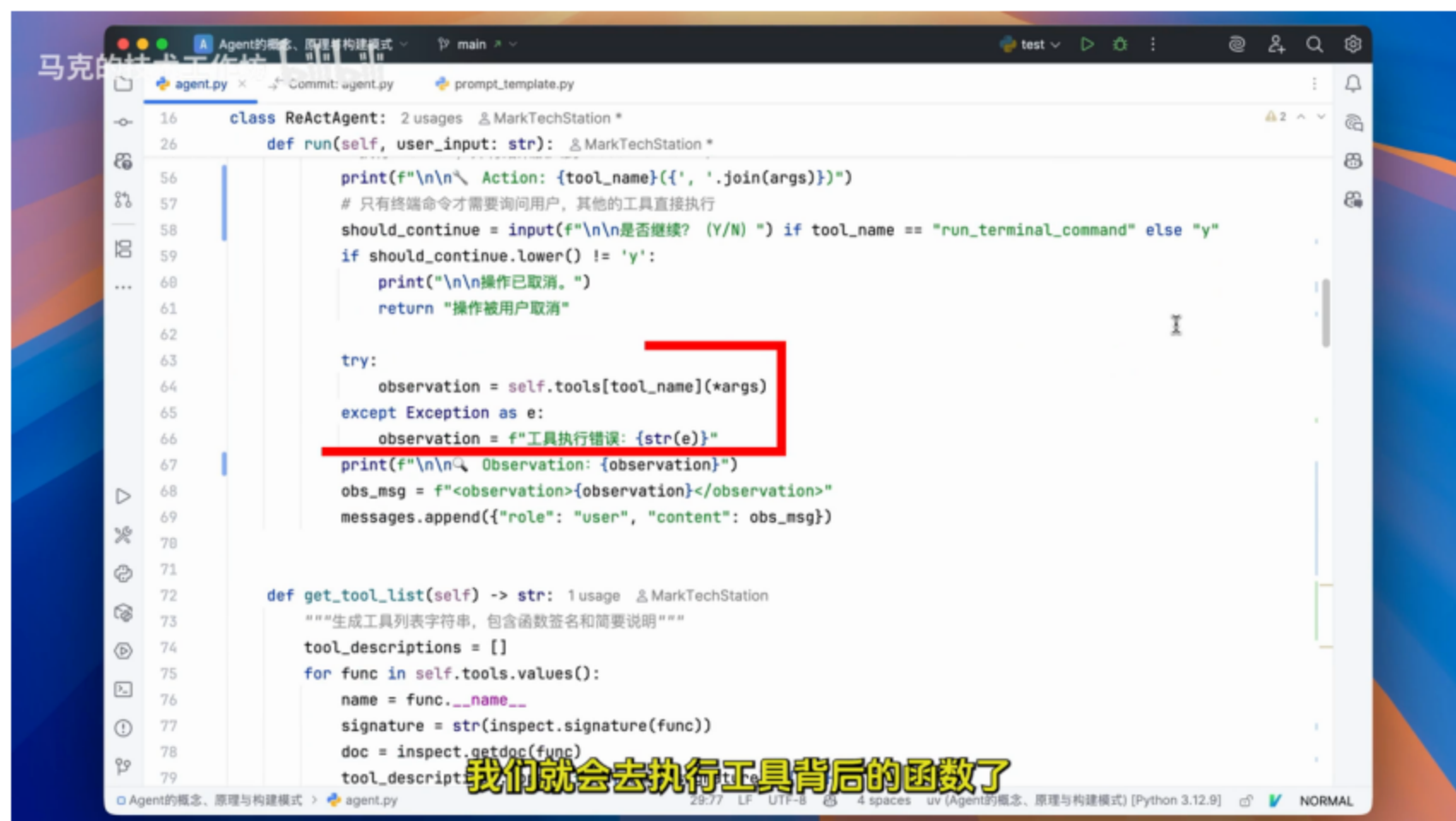


图 7-5 run() 主循环：call_model → 解析 thought/action → 执行工具 → 回填 observation

八、Agent 架构全景图

为了彻底看清整个问答过程中发生了什么，视频画了一张流程图。流程里有两类角色：用户和 Agent；而 Agent 又可分为三部分：

- 模型（大脑，负责思考与决定 Action）；
- 工具（函数，真正干活的四肢）；
- Agent 主程序（负责串联整个流程的代码逻辑，可以理解为前面代码里的 run 函数）。

它们的沟通方式是：

- 用户提交任务 → 任务先到 Agent 主程序；
- 主程序调用模型 → 模型返回 Thought 和 Action；
- 主程序把 Thought/Action 展示给用户，然后去调用 Action 指定的工具；
- 工具执行完返回结果 → 主程序把结果展示给用户，并加入历史消息列表；
- 再次请求模型，重复上述「思考→行动→观察」直至模型返回 Thought + Final Answer，流程结束。

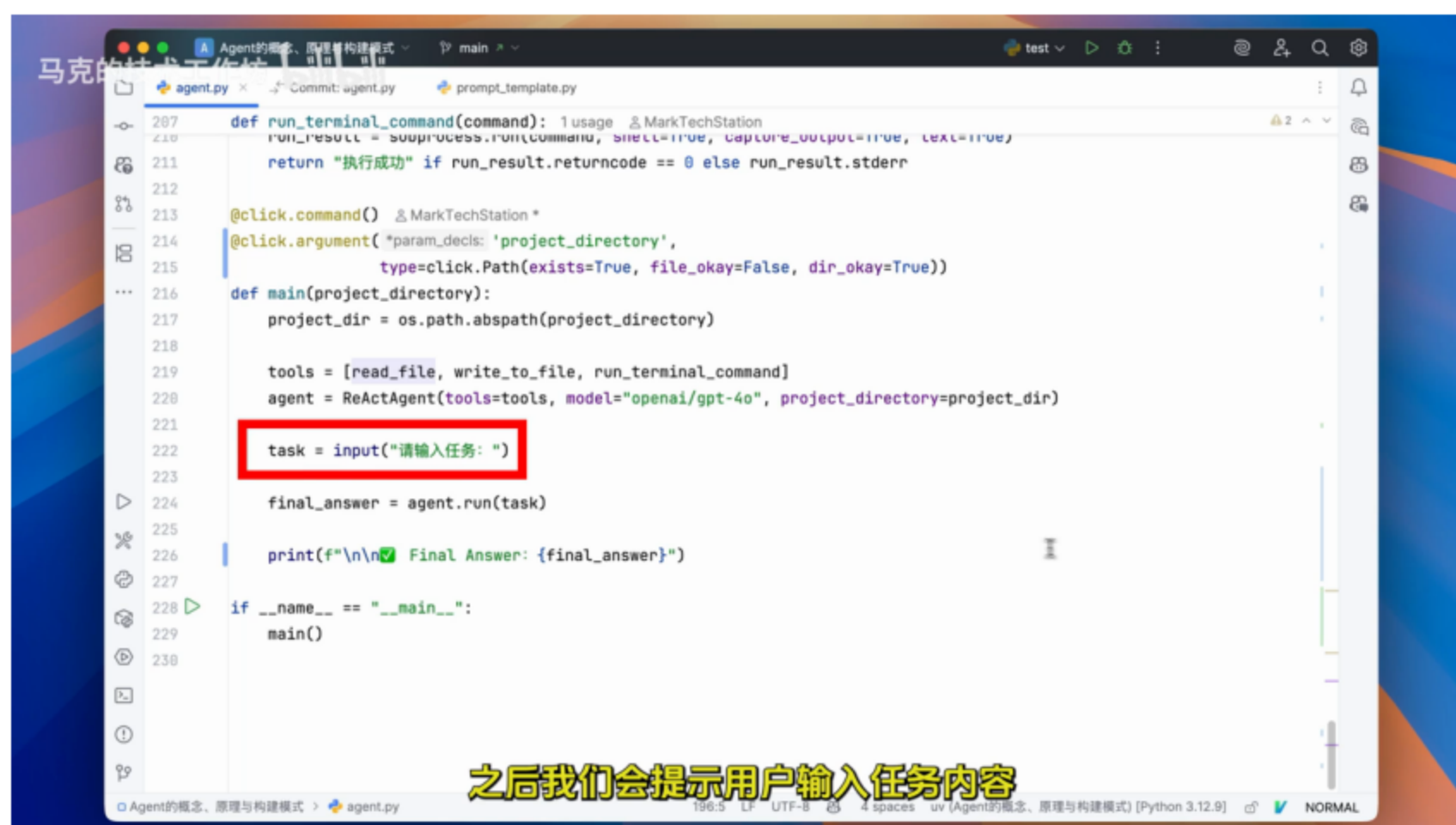


图 8-1 Agent 架构全景：用户 / 主程序 / 模型 / 工具 四方协作

九、另一种运行模式：Plan & Execute（先规划，再执行）

ReAct 最常见、使用最广，但它不是唯一方案。很多 Agent 的运行方式是「先规划，再执行」。

- Manus 一开始就构建一个待办列表，后续都按列表执行；
- Claude Code 里也常看到先创建 to-do、再去执行的情况。

这种「先规划再执行」目前没有统一名字，各实现也略有差异。视频重点讲了其中一个有名的实践——LangChain 提出的 Plan & Execute 模式。

9.1 模式结构

从总体看它也遵循「先规划再执行」，但引入了「动态调整规划」的环节，灵活性更强。其内部模块有：

- Plan 模型：负责生成执行计划；
- Replan 模型：根据每步执行结果动态调整计划（plan/replan 可以是同一个模型，也可分开）；
- 执行 Agent：负责执行计划中的每一步（它内部仍可用 ReAct 模式运行，内置如网络搜索等工具）；

- Agent 主程序：负责串联整个流程。

注意：Plan & Execute Agent 内部还套着一个 Agent（执行 Agent），这种「Agent 套 Agent」的设计在实践中很常见。

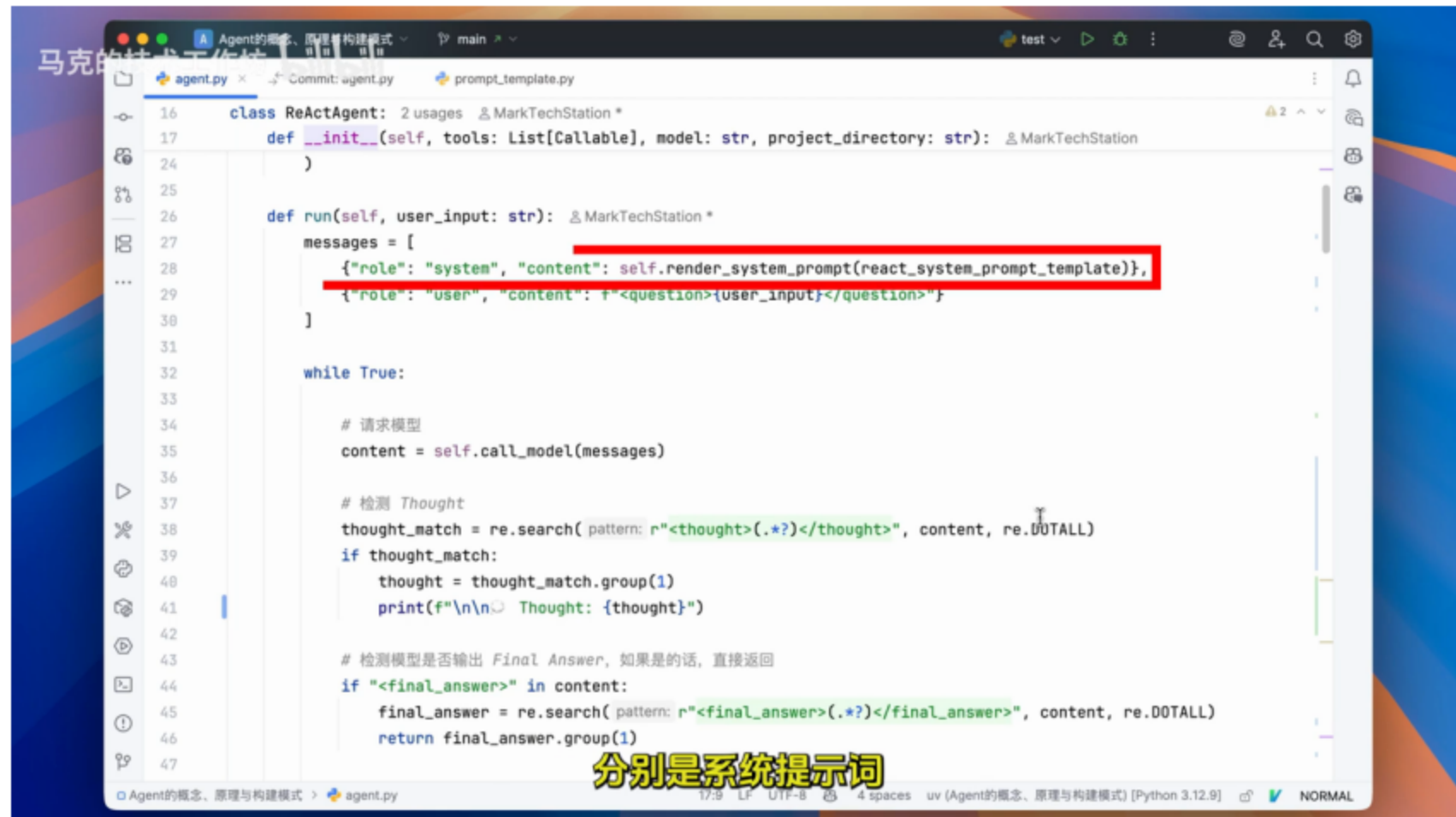


图 9-1 Plan & Execute 思路：先规划，再执行（以 Claude Code 的 to-do 为例）

9.2 角色与时序

以「今年澳网男子冠军的家乡在哪里」为例，主程序把问题发给 Plan 模型，得到执行步骤（查询当前日期 → 查询该年澳网男单冠军名字 → 据名字查家乡）。随后主程序把计划交给执行 Agent 执行第一步，执行完后把「用户问题 + 计划 + 执行记录」发给 Replan 模型，生成下一轮计划。

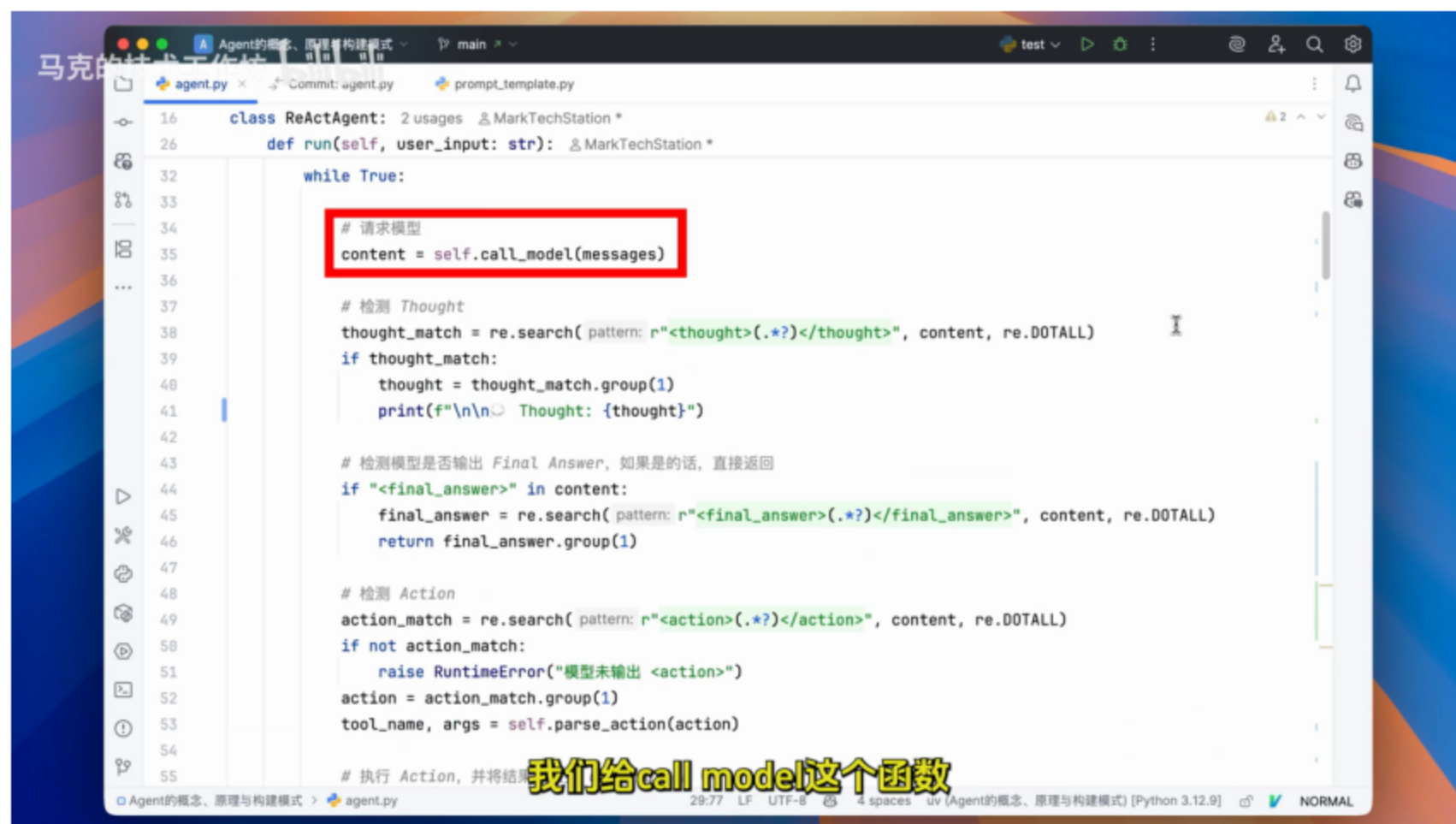


图 9-2 Plan & Execute 模块：Plan 模型 / Replan 模型 / 执行 Agent / 主程序

9.3 三轮循环模拟

整个循环一般运行多轮，以题为例运行三轮，对应计划中的三步。每一轮含「执行环节」+「Replan 环节」：

- 第一轮：执行「查询当前日期」→ Replan 模型更新计划（已查日期，第二步精确到「2025 年澳网男单冠军」）；
- 第二轮：执行「查冠军名字」→ Replan 再次更新；
- 第三轮：执行「查家乡」→ 此时所有步骤完成，Replan 模型发现任务已可作答，便不再返回新计划，而是直接返回「最终答案」。

因此更准确地说：主程序请求 Replan 模型，返回的是「新的执行计划」或「最终答案」二选一——有步骤要做就给计划，没步骤要做就给答案。

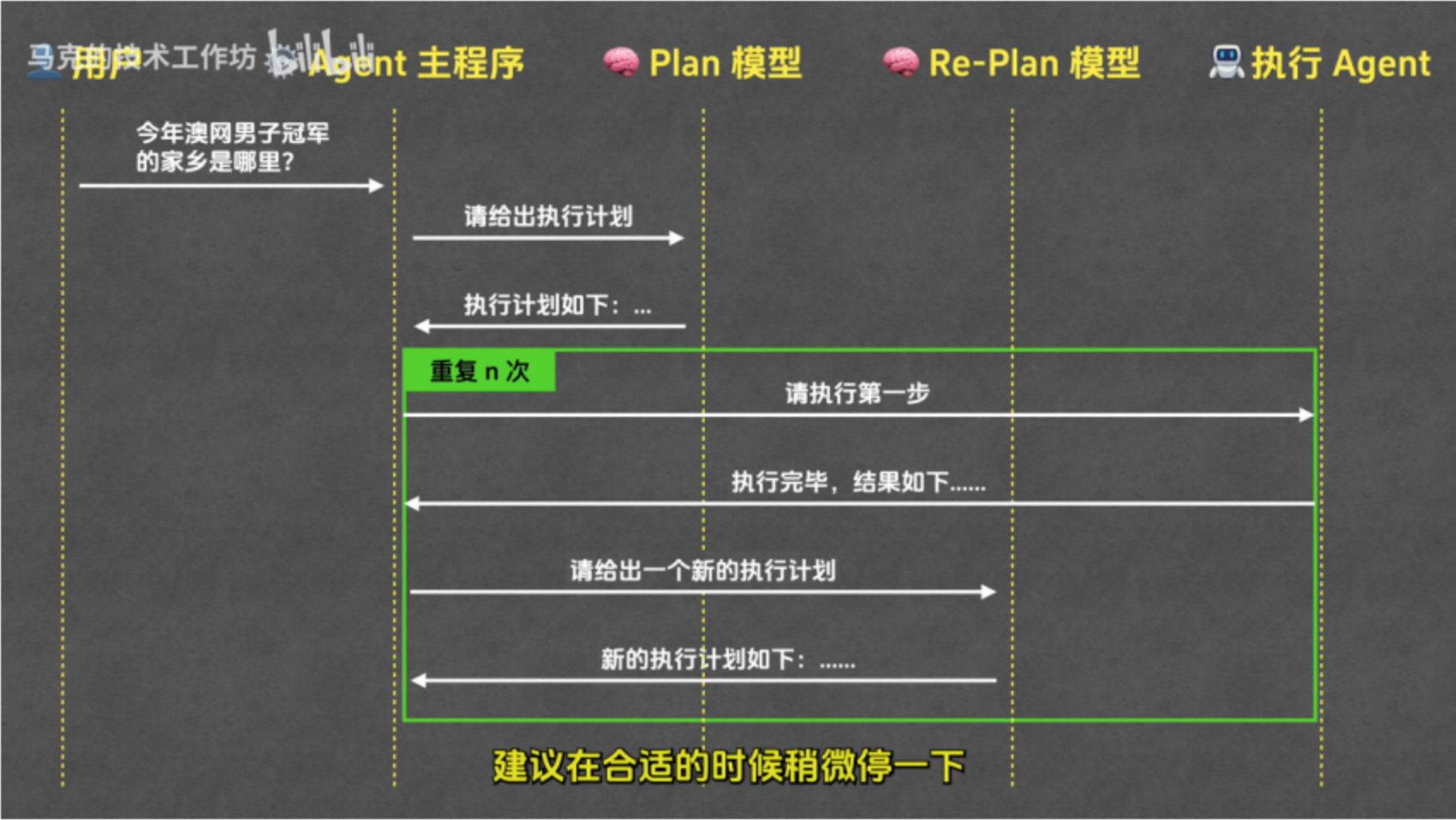


图 9-3 三轮循环：执行 → Replan，直到所有步骤完成返回最终答案

十、两种模式对比

把 ReAct 与 Plan & Execute 放在一起看，更容易理解各自的取舍：

对比维度	ReAct	Plan & Execute
核心思路	边思考边行动，循环推进	先整体规划，再分步执行并可动态调整
提出/代表	2022.10 论文；最广泛使用	LangChain 实践方案
规划方式	隐式，在 Thought 中逐步决定	显式，由 Plan/Replan 模型维护计划
适用场景	步骤不固定、需边做边看的任务	目标清晰、可预先拆解的复杂任务
灵活性	灵活但易走弯路	计划清晰，且能按结果动态修正
典型产品	Cursor、简化版 Claude Code	Manus、Claude Code(to-do 模式)

十一、总结与学习建议

回到开头的两个问题——Agent 是什么、怎么运作的，现在应该很清楚了：

- 是什么：把「大模型」和「工具」组装起来，能感知并改变外界环境的智能程序；
- 怎么运作：靠运行模式驱动，最经典的是 ReAct（思考→行动→观察→循环→最终答案），靠系统提示词「安排剧本」，靠 Agent 主程序串联模型与工具；进阶做法是 Plan & Execute（先规划再执行、动态修正）。

视频作者也给出了一条务实的学习路径：

- 先把 ReAct 的论文流程和四要素（Thought/Action/Observation/Final Answer）记牢；
- 动手跑一遍开源的简化版 Agent，逐行读 main()、ReactAgent、run() 循环；
- 理解了 ReAct 之后，再看 Plan & Execute 等其它模式会更轻松；
- 刻意区分「模型请求工具」与「工具调用组件真正执行工具」——这是很多初学者最容易混淆的点。

至此，你已经打通了从「大模型能力边界」到「自己写出一个简化版 Claude Code」的完整链路。点赞关注，下次再见。



图 11-1 课程收尾：从概念到动手实现一个 Agent